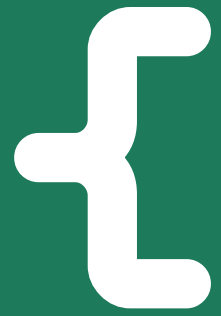
Podstawy programowania w Java

mgr inż. Sławomir Aleksander Dańczak

Katedra Metod Programowania
Polsko-Japońska Akademia Technik Komputerowych



JAVA

Instrukcja iteracyjna, zaniechania i kontynuacji, typ wyliczeniowy

Wykład 04

Plan na dzisiaj...

00

INSTRUKCJA ITERACYJNA → INSTRUKCJA ZANIECHANIA I KONTYNUACJI → TYP WYLICZNIKOWY

Instrukcja iteracyjna 01

Jak sprawić, żeby program wykonywał coś wielokrotnie?

WARUNEK → WYKONANIE → POWTÓRZENIE

Instrukcja iteracyjna

Instrukcje iteracyjne umożliwiają cykliczne wykonywanie instrukcji.

Instrukcja iteracyjna powinna gwarantować zakończenie jej wykonania

Dostępne instrukcje iteracyjne:

- pętla while
- pętla do-while
- pętla for
- pętla for-each

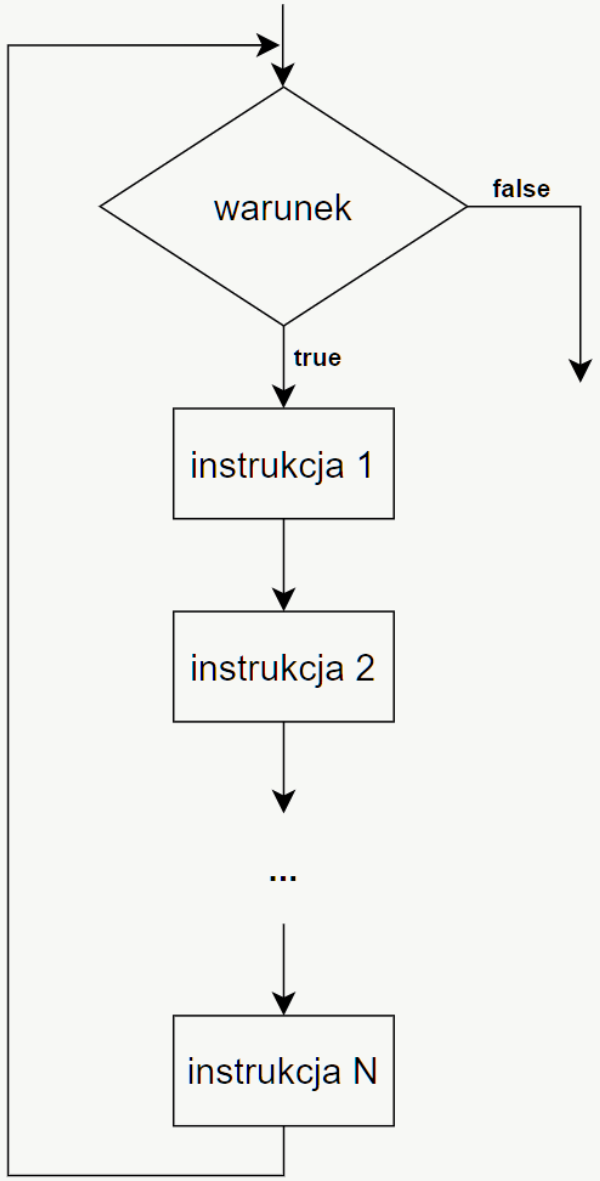
Pętla while

Instrukcja **while** ma postać:

```
while(warunek)
    instrukcja;


lub


while(warunek){
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
}
```



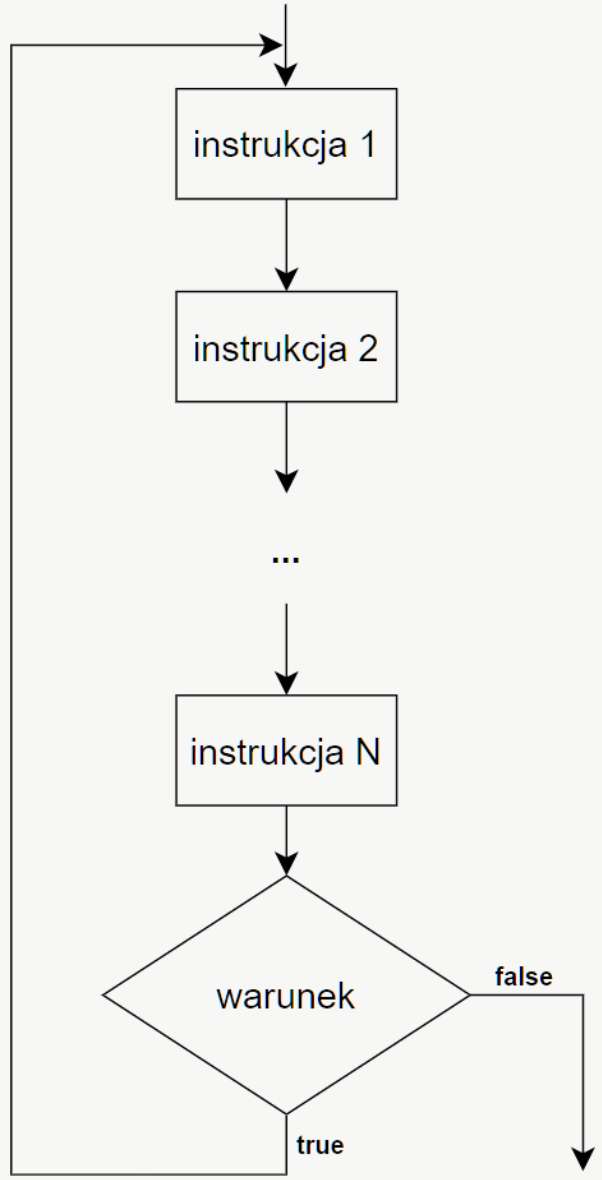
Pętla do-while

Instrukcja **do-while** ma postać:

```
do
    instrukcja;
while(warunek);
```

lub

```
do{
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
}while(warunek);
```



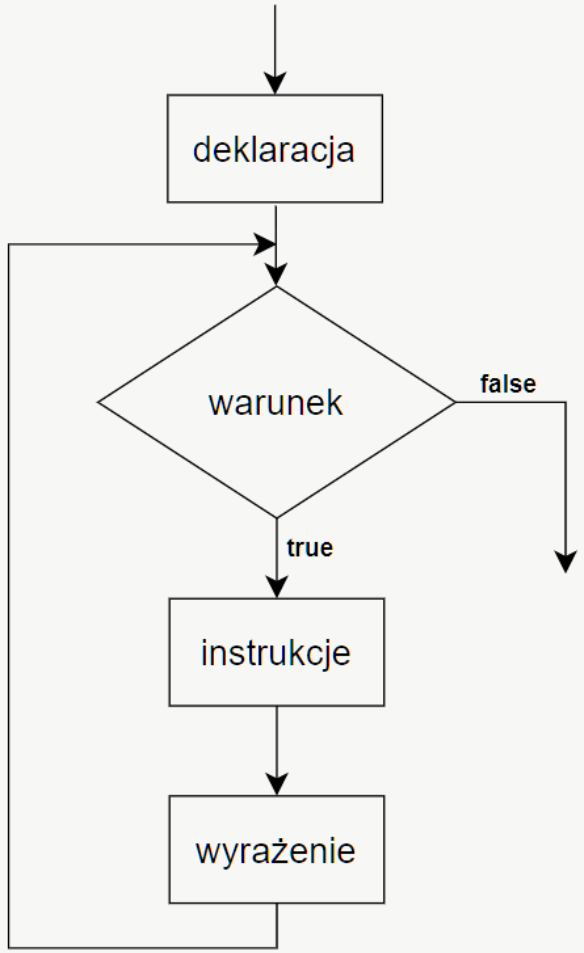
Pętla for

Instrukcja **for** ma postać:

```
for (deklaracja; warunek; wyrażenie)
    instrukcja;

lub

for (deklaracja; warunek; wyrażenie){
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
}
```



Pętla for-each

Instrukcja **for-each** ma postać:

```
for (deklaracja : zbiór_danych)
    instrukcja;
```

lub

```
for (deklaracja : zbiór_danych){
    instrukcja1;
    instrukcja2;
    ...
    instrukcjaN;
}
```

Instrukcja zaniechania i kontynuacji

02

Jak przejąć kontrolę?

BREAK → CONTINUE → KONTROLA

Instrukcja zaniechania i kontynuacji

We wnętrzu pętli mogą być użyte instrukcje

```
break;
```

oraz

```
continue;
```

Wykonanie instrukcji **zaniechania (break)** powoduje zakończenie wykonania pętli.

Wykonanie instrukcji **kontynuowania (continiue)** powoduje potraktowanie jeszcze nie wykonanych instrukcji bieżącego obrotu pętli jak instrukcji pustych (nie zostaną wykonane i przejdziemy do kolejnego obrotu pętli).

Typ wyliczeniowy

03

Jak reprezentować zamknięty zbiór możliwych wartości?

ZBIÓR WARTOŚCI → ENUM → BEZPIECZEŃSTWO → CZYTELNOŚĆ

Typ wyliczeniowy

Typ wyliczeniowy jest specjalnym typem danych, który reprezentuje zamknięty zestaw stałych, nazwanych wartości, które mogą być używane jako symbole lub etykiety w programie. Typy wyliczeniowe są przydatne, gdy chcemy ograniczyć zestaw możliwych wartości i jednocześnie uczynić kod bardziej czytelnym.

```
enum WeekDay{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
}
```

Typ wyliczenikowy

```
enum Swiatlo { CZERWONE, ZOLTE, ZIELONE }

Swiatlo sygnal = Swiatlo.ZIELONE;

if (sygnal == Swiatlo.ZIELONE)
    System.out.println("Jedź");

// Swiatlo x = "ZIELONE"; // błąd kompilacji
```

Zadania wykładowe

04

01 Napisz program, który za pomocą pętli for wyświetli wszystkie liczby parzyste z przedziału od 1 do 50. Zrealizuj zadanie na trzy sposoby, wykorzystując kolejno pętle **while**, **do-while** oraz **for**

02 Zdefiniuj typ wyliczeniowy **enum** reprezentujący poziomy trudności gry: **LATWY**, **SREDNI** i **TRUDNY**.
Utwórz zmienną tego typu i przypisz jej wybrany poziom trudności. Następnie za pomocą instrukcji warunkowej **if** wyświetl odpowiednią liczbę punktów początkowych:
LATWY - 100 punktów
SREDNI - 50 punktów
TRUDNY - 10 punktów

03 Napisz program, który za pomocą pętli **while** obliczy sumę wszystkich liczb od 1 do N

Pytania?

05

> waiting for input...

Dziękuję za uwagę

06

THE END